PROJECT DOCUMENTATION

Comment Section Polarization Analyzer

A Transformer-Based NLP System for Social Media Sentiment Analysis

Prepared By

Sadiya Siddiqui Kausar

Roll No.: 27

Masters Programme | Department of Computer Science

Academic Year 2025-2026

Table of Contents

1. **Introduction** 3
   - 1.1 Project Overview 3
   - 1.2 Motivation 3
   - 1.3 Objectives 3
   - 1.4 Scope 4
2. **System Architecture** 5
   - 2.1 High-Level Architecture 5
   - 2.2 Component Interaction 6
3. **Module Descriptions** 7
   - 3.1 Data Acquisition Module 7
   - 3.2 Analysis Engine 7
   - 3.3 User Interface 9
4. **Key Design Decisions and Novelty** 10
5. **Technology Stack** 11
6. **Data Flow and Processing Pipeline** 12
7. **Configuration and Environment Setup** 14

1. Introduction

1.1 Project Overview

The Comment Section Polarization Analyzer is an end-to-end Natural Language Processing (NLP) application designed to automatically retrieve, classify, and quantify the sentiment distribution within online comment sections. The system leverages a state-of-the-art transformer-based language model specifically fine-tuned on social media data, to perform robust multi-class sentiment analysis on raw user comments fetched directly from YouTube via the official YouTube Data API v3.

The application goes beyond traditional binary sentiment classification by introducing a confidence-aware adjustment mechanism and a novel polarization score metric, providing analysts with both qualitative and quantitative insight into the nature of audience opinion on any given piece of content.

1.2 Motivation

Understanding how audiences respond to online content has become an increasingly critical challenge in the domains of computational social science, digital marketing, and public opinion research. Standard sentiment tools built on general-domain corpora often underperform on informal, emoji-laden, and slang-heavy social media text. This project directly addresses that limitation by employing a social-media-trained transformer model and augmenting its output with rule-based heuristics for expressive language.

Furthermore, while existing tools typically present only a raw count of positive and negative comments, they fail to quantify the degree of divergence in audience opinion -- a concept central to the study of online polarization. This project introduces a dedicated polarization score to fill this analytical gap.

1.3 Objectives

- To build a functional pipeline for automated retrieval of YouTube comments using the YouTube Data API v3.
- To perform accurate, multi-class sentiment classification using the cardiffnlp/twitter-roberta-base-sentiment transformer model.
- To implement a confidence-based filtering mechanism that reduces erroneous classifications from low-certainty model predictions.
- To detect expressive, informal positive sentiment (e.g., slang, emojis) through a hybrid rule-based and model-based approach.
- To compute and present a numerical polarization score that quantifies the divergence between positive and negative audience opinion.

- To deliver results through an intuitive, interactive web dashboard built with Streamlit and Plotly.

1.4 Scope

The current implementation supports YouTube as the primary data source, with the architecture designed to be easily extensible to other platforms such as Reddit or Twitter/X. The system processes up to 100 top-level comments per query, which is the maximum permitted per request under the standard YouTube Data API v3 quota.

2. System Architecture

The application follows a modular, three-tier architecture separating data acquisition, analytical processing, and user interface presentation. This separation of concerns ensures maintainability, testability, and ease of extension.

2.1 High-Level Architecture

The data flow through the system proceeds as follows: the user submits a YouTube video URL via the Streamlit frontend; the Data Acquisition module resolves and validates the video ID before invoking the YouTube Data API; the retrieved raw comments are passed to the Analysis Engine; results are aggregated into a structured DataFrame along with the computed polarization score; and finally, the Visualization Layer renders the findings as an interactive dashboard.

Module / File	Technology	Role / Responsibility
app.py	Python / Streamlit	Main application entry point; manages UI state, user inputs, and orchestrates the full pipeline from data fetch to result rendering.
youtube_fetch.py	Python / Google API Client	Handles YouTube URL parsing, video ID extraction, and comment retrieval via the YouTube Data API v3.
bert_analysis.py	Python / Hugging Face Transformers	Core NLP engine; loads the RoBERTa model, performs sentiment inference, applies confidence filtering, detects expressive praise, and computes the polarization score.

2.2 Component Interaction

The Streamlit application (app.py) acts as the controller layer, importing and calling functions from both subordinate modules. The youtube_fetch module is invoked first to populate the comments list; this list is then passed directly to the bert_analysis module. All output -- the annotated DataFrame and the polarization score -- is returned to the controller for display. This design ensures that neither subordinate module has any dependency on Streamlit, making them independently testable and reusable in non-UI contexts.

3. Module Descriptions

3.1 Data Acquisition Module -- youtube_fetch.py

3.1.1 Purpose

This module encapsulates all communication with the YouTube Data API v3 and provides URL parsing utilities. It is the sole interface between the application and external YouTube data.

3.1.2 Function: extract_video_id(youtube_url)

Accepts a raw YouTube URL string in any of the following canonical formats and extracts the eleven-character video identifier using regular expression pattern matching:

- Standard watch URL: https://www.youtube.com/watch?v=VIDEO_ID
- Shortened URL: https://youtu.be/VIDEO_ID
- YouTube Shorts URL: https://www.youtube.com/shorts/VIDEO_ID

A ValueError is raised if none of the recognised patterns match the provided URL, preventing invalid API calls downstream.

3.1.3 Function: get_youtube_comments(video_id, max_comments=100)

Builds a YouTube service object using the googleapiclient library, authenticating via an API key read securely from the YOUTUBE_API_KEY environment variable. The function queries the commentThreads endpoint, requesting the snippet part with plain text formatting, and returns a flat Python list of top-level comment strings. Cache discovery is explicitly disabled to avoid file system warnings in deployment environments.

The max_comments parameter is capped at 100 per request, consistent with the YouTube API's per-page maximum. Future work may incorporate pagination (nextPageToken) to support larger corpora.

3.2 Analysis Engine -- bert_analysis.py

3.2.1 Purpose

This module constitutes the analytical core of the system. It is responsible for sentiment classification, confidence-aware post-processing, expressive language detection, and polarization score computation.

3.2.2 Model Selection: cardiffnlp/twitter-roberta-base-sentiment

The chosen model is a RoBERTa-base architecture fine-tuned on the TweetEval benchmark dataset (Barbieri et al., ACL 2020), consisting of over 58 million tweets. This training corpus makes the model significantly more robust to informal text patterns -- including abbreviations, hashtags, and code-switching -- compared to models trained exclusively on formal corpora. The model predicts one of three classes: LABEL_0 (Negative), LABEL_1 (Neutral), and LABEL_2 (Positive), along with a softmax confidence score.

3.2.3 Expressive Praise Detection: contains_expressive_praise(text)

A domain-specific rule-based function that inspects comment text for the presence of curated lists of culturally prevalent positive slang terms and positive emojis. The function returns a Boolean flag. This heuristic is applied specifically to comments that the RoBERTa model classifies as Neutral with a confidence score exceeding 0.70, reclassifying them as positive-expressive.

Praise word lexicon includes: love, slay, slayed, slaying, queen, king, fire, lit, iconic, legend, amazing, perfect, obsessed, goat, best, ate.

Praise emoji set includes: fire, heart, heart-eyes, folded-hands, crown, laughing, and crying-face emojis.

3.2.4 Function: analyze_comments(comments)

The main analysis function accepts a list of comment strings and returns a tuple comprising an annotated pandas DataFrame and a floating-point polarization score. For each comment, the text is truncated to 512 tokens (the model's maximum context length) before inference; the raw LABEL output is mapped to a human-readable sentiment class via the LABEL_MAP dictionary; the expressive praise heuristic is applied where applicable; and all results are accumulated into a list of dictionaries before DataFrame construction.

A safety guard at the function entry point handles the edge case of an empty input list, returning an empty DataFrame and a score of 0.0 to prevent downstream errors.

3.2.5 Polarization Score Computation

The polarization score is defined as the normalised absolute difference between the count of positive-class comments (both positive and positive-expressive) and the count of negative-class comments, relative to the total number of comments:

```
polarization_score = |count(positive) - count(negative)| / total_comments
```

This metric ranges from 0.0 (perfectly balanced positive and negative sentiment, indicating maximum divergence) to 1.0 (all comments belong to a single class, indicating consensus). A lower score therefore indicates greater polarization, while a higher score indicates stronger directional consensus.

3.3 User Interface -- app.py

3.3.1 Purpose

The Streamlit application serves as the presentation and interaction layer, providing a browser-based interface requiring no client-side installation.

3.3.2 Confidence Threshold Slider

A slider widget allows the user to set a confidence threshold between 0.0 and 1.0 (default: 0.6). After primary model inference and expressive praise detection, a post-processing step reclassifies any comment whose model confidence falls below this threshold as Neutral. This sentiment_adjusted column is used for all downstream visualisations, allowing the user to experiment with the trade-off between recall (lower threshold) and precision (higher threshold) without re-running the model.

3.3.3 Visualisation Components

- Metrics Row: Two Streamlit metric cards display the polarization score (rounded to three decimal places) and the total number of comments analysed.
- Expandable Data Table: An st.expander widget exposes the full annotated DataFrame, showing raw comment text, raw model sentiment, confidence score, and adjusted sentiment for each record.
- Sentiment Pie Chart: An interactive Plotly pie chart visualises the distribution of sentiment categories using a semantically meaningful colour palette (green for positive, blue for positive-expressive, grey for neutral, red for negative).

4. Key Design Decisions and Novelty

This section articulates the specific design choices that distinguish this project from a standard off-the-shelf sentiment classifier.

4.1 Social-Media-Specific Model

The deliberate selection of cardiffnlp/twitter-roberta-base-sentiment over a generic BERT-base model reflects an understanding that domain shift is a significant performance degrader in NLP. General-domain models trained on Wikipedia or BookCorpus struggle with the orthographic irregularities, neologisms, and emotional intensity characteristic of platform comment sections. The Twitter-trained RoBERTa model mitigates this domain mismatch.

4.2 Dual Sentiment Representation

The system maintains both a raw sentiment column (direct model output) and an adjusted sentiment column (post-processed with the confidence threshold). This dual representation preserves the original model predictions for audit purposes while surfacing a more reliable signal for analytical reporting, consistent with best practices in responsible AI deployment.

4.3 Confidence-Based Neutralisation

Rather than accepting all model outputs unconditionally, the system applies a user-configurable confidence threshold. Predictions made with low certainty are demoted to the Neutral class, reducing the rate of false positive and false negative assignments. This is particularly important for short or ambiguous comments where the model's softmax probability distribution may be nearly uniform.

4.4 Positive-Expressive as a Distinct Category

The introduction of a fourth sentiment class -- positive-expressive -- acknowledges the pragmatic reality of social media communication. Comments such as "she ATE this [fire emojis]" or "absolute QUEEN [heart-eyes emoji]" convey strong positive sentiment through cultural codes that a neutral-trained model may not fully capture. By explicitly labelling this category, the system provides finer-grained insight into the nature of positive audience reactions.

4.5 Custom Polarization Score

The custom polarization metric provides a single, interpretable number summarising the overall directional tension in the comment section. This has direct utility for content creators, social researchers, and brand analysts who need a quick, comparable summary statistic across multiple videos or time periods.

5. Technology Stack

Component	Technology / Library	Notes
Programming Language	Python 3.x	Core implementation language
Web Framework	Streamlit	Interactive web dashboard
NLP Model	Hugging Face Transformers	cardiffnlp/twitter-roberta-base-sentiment
Data Manipulation	pandas	DataFrame operations and result aggregation
Visualization	Plotly Express	Interactive pie charts and distributions
YouTube API Client	google-api-python-client	YouTube Data API v3 integration

6. Data Flow and Processing Pipeline

The following describes the complete end-to-end processing pipeline from user input to final output.

Step 1 -- URL Input and Validation

The user enters a YouTube video URL in the Streamlit text input field and selects a confidence threshold via the slider widget. Upon clicking Fetch & Analyze, the extract_video_id() function is called. If the URL does not match any recognised pattern, an error is displayed and processing halts. Valid URLs yield an eleven-character video ID.

Step 2 -- Comment Retrieval

The video ID is passed to get_youtube_comments(), which constructs an authenticated YouTube API service object. A commentThreads.list() request is issued, and the plain text content of each top-level comment is extracted from the API response.

Step 3 -- Sentiment Inference

The comment list is passed to analyze_comments(). For each comment, the text is truncated to 512 characters and fed to the RoBERTa pipeline. The returned LABEL and score are mapped to human-readable sentiment and confidence values respectively.

Step 4 -- Expressive Praise Detection

For any comment classified as Neutral with a confidence score above 0.70, the contains_expressive_praise() function is invoked. If positive slang terms or emojis are detected, the sentiment is overridden to positive-expressive.

Step 5 -- Polarization Score Computation

After all comments are processed, the polarization score is computed as the normalised absolute difference between positive-class and negative-class comment counts.

Step 6 -- Confidence Threshold Application

A vectorised pandas operation applies the user-specified confidence threshold, producing a sentiment_adjusted column. Any comment with model confidence below the threshold is reclassified as Neutral.

Step 7 -- Rendering

The Streamlit application renders the metric cards, expandable data table, and Plotly pie chart using the adjusted sentiment values, completing the user-facing output.

7. Configuration and Environment Setup

7.1 Environment Variables

The application reads sensitive credentials exclusively from environment variables to avoid hard-coding secrets in source code. The following variable must be set prior to running the application:

Variable Name	YOUTUBE_API_KEY
Description	A valid API key for the YouTube Data API v3, obtained from the Google Cloud Console.
Setting (Unix/macOS)	export YOUTUBE_API_KEY="your_api_key_here"
Setting (Windows)	set YOUTUBE_API_KEY=your_api_key_here

7.2 Installation

All required Python dependencies can be installed using pip. It is strongly recommended to use a dedicated virtual environment (venv or conda) to avoid dependency conflicts, particularly between PyTorch and Hugging Face libraries.

```
pip install streamlit pandas plotly transformers torch google-api-python-client
sentence-transformers scikit-learn
```

7.3 Running the Application

```
streamlit run app.py
```

The Streamlit development server will launch the application in the default web browser at http://localhost:8501.